

EVALUACIÓN PA04

Recursividad, principio divide y vencerás, algoritmos recursivos de ordenamiento, grafos, DFS, BFS, A* y aplicaciones básicas en IA

Curso: Algoritmo y Estructura de Datos Basados en Inteligencia Artificial

Docente: WILDER JULIO ESPINOZA BRAVO

Integrantes:

- **Diego Ricardo Arcos Espinoza - 100%**
 - **Jerson Herrera Salazar – 100%**
- **Fabian Marcelo Davila Herrera – 100%**
 - **Juan José Montenegro - 100%**

Índice:

1. Actividad 1, Recursividad y trazabilidad del algoritmo
2. Actividad 2, Ordenamiento recursivo aplicado
3. Actividad 3, Modelado del problema con grafos y búsquedas
4. Actividad 4, Sustento técnico y cierre grupal

Actividad 1

Un algoritmo recursivo es aquel que resuelve un problema llamándose a sí mismo una o varias veces, reduciendo el tamaño del problema en cada llamada hasta llegar a una condición de parada llamada caso base, esta se relaciona con el principio Divide y Vencerás porque divide un problema grande en problemas más pequeños del mismo tipo, en donde cada llamada resuelve una parte del problema y, al finalizar todas las llamadas, se obtiene la solución completa.

Piden calcular el número total de pasos necesarios para revisar una secuencia de puntos de control del campus:

Algoritmo ContarPasos

Funcion RevisarPuntos(puntos)

Si puntos = 0 Entonces

Retornar 0

Fin Si

Retornar 1 + RevisarPuntos(puntos - 1)

FinFuncion

Inicio

Escribir "Ingrese la cantidad de puntos de control:"

Leer puntos

total <- RevisarPuntos(puntos)

Escribir "Total de pasos de revisión: ", total

Fin

Ahora piden la traza manual de ejecución:

RevisarPuntos(5)

= 1 + RevisarPuntos(4)

= 1 + (1 + RevisarPuntos(3))

= 1 + (1 + (1 + RevisarPuntos(2)))

= 1 + (1 + (1 + (1 + RevisarPuntos(1))))

= 1 + (1 + (1 + (1 + (1 + RevisarPuntos(0)))))

RevisarPuntos(0) = 0

Y el retorno es:

RevisarPuntos(0) = 0

RevisarPuntos(1) = 1

RevisarPuntos(2) = 2

RevisarPuntos(3) = 3

RevisarPuntos(4) = 4

RevisarPuntos(5) = 5

Total de pasos de revisión = 5

Esta ejecución presenta 6 llamadas recursiva, que vendrían a ser los 5 niveles significativos más el caso base.

Un caso base es cuando no quedan puntos por revisar, digámosle como el fin.

El caso recursivo ocurre porque mientras existan puntos por revisar, se cuenta uno y se vuelve a llamar a la función con un punto menos.

Y para el último; el criterio de finalización, en donde el algoritmo termina cuando la cantidad de puntos llega a 0, evitando llamadas recursivas infinitas y permitiendo que comiencen los retornos hasta obtener el resultado final.

Actividad 2

tiempos = [12, 7, 25, 4, 18, 9, 15, 2]

Elegimos Merge Sort porque es el ejemplo más representativo del principio divide y vencerás: parte el problema en mitades, ordena cada mitad por separado de forma recursiva, y al final las combina (merge) en un solo arreglo ordenado. Esto lo hace ideal para mostrar con claridad cómo funciona la recursividad aplicada al ordenamiento.

División. El algoritmo toma el arreglo y lo parte en dos mitades. Luego vuelve a dividir cada mitad. Y otra vez. Y otra. Hasta que cada parte tiene un solo elemento (caso base: un arreglo de 1 elemento está, por definición, ordenado).

División del vector [12, 7, 25, 4, 18, 9, 15, 2]:

```

      [12, 7, 25, 4, 18, 9, 15, 2]
        /      \
    [12, 7, 25, 4]  [18, 9, 15, 2]
      /  \      /  \
    [12, 7] [25, 4] [18, 9] [15, 2]
    /\   /\   /\   /\
  [12][7] [25][4]  [18][9] [15][2]
```

Combinación (merge). Una vez que cada mitad está dividida hasta el mínimo, se empiezan a fusionar de vuelta comparando los elementos. En cada fusión, se recorren las dos mitades en paralelo y se va eligiendo el menor en cada paso, lo que produce una mitad combinada ya ordenada.

Combinación ascendente:

```

      [12][7] → [7,12]   [25][4] → [4,25]   [18][9] → [9,18]   [15][2]
→ [2,15]
```

$[7,12] + [4,25] \rightarrow [4, 7, 12, 25]$ $[9,18] + [2,15] \rightarrow [2, 9, 15,$
18]

$[4, 7, 12, 25] + [2, 9, 15, 18] \rightarrow [2, 4, 7, 9, 12, 15, 18, 25]$

¿Por qué la estrategia recursiva es válida aquí?

La recursividad es natural en este problema porque el problema **se puede expresar en términos de sí mismo**: ordenar un arreglo grande equivale a ordenar dos arreglos más pequeños y luego fusionarlos. Esto es exactamente el principio **divide y vencerás**:

- **Dividir** el problema grande en subproblemas más pequeños del mismo tipo.
- **Vencer** cada subproblema resolviéndolo recursivamente (o directamente si es trivial, el caso base).
- **Combinar** las soluciones parciales para obtener la solución final.

Análisis breve de eficiencia

La complejidad de este algoritmo (Merge Sort) es **$\log n$** tanto en el mejor como en el peor caso, lo cual lo hace muy estable y predecible. La intuición es la siguiente:

- **$\log n$ niveles de división.** Cada vez que se divide el arreglo a la mitad, su tamaño se reduce a la mitad; con 8 elementos hay 3 niveles de división ($8 \rightarrow 4 \rightarrow 2 \rightarrow 1$).
- **n operaciones por nivel.** En cada nivel, al fusionar todas las mitades, se tocan en total los n elementos del arreglo.

Actividad 3

Actividad 3: Modelado del problema con grafos y búsquedas

Modelado del campus

Para representar el campus utilizamos un **grafo no dirigido**, ya que los estudiantes pueden desplazarse en ambos sentidos entre los diferentes ambientes.

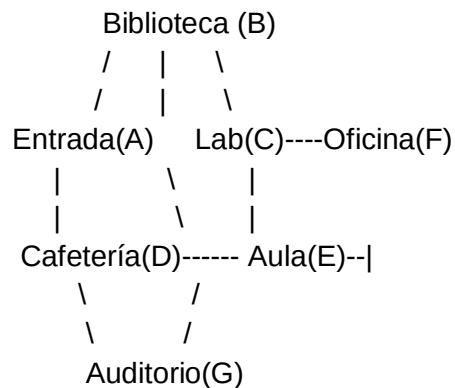
Nodos

- A = Entrada Principal
- B = Biblioteca
- C = Laboratorio de Informática
- D = Cafetería
- E = Aula 101
- F = Oficina Administrativa
- G = Auditorio

Conexiones

1. A – B
2. A – D
3. B – C
4. B – E
5. C – E
6. C – F
7. D – E
8. D – G
9. E – G
10. F – G

Grafo



El grafo es **no dirigido** porque se puede ir y regresar entre los ambientes conectados.

Recorrido DFS

Tomando como punto de inicio la Entrada Principal (A), el recorrido obtenido es:

A → B → C → E → D → G → F

Este algoritmo explora un camino completo antes de regresar para visitar los demás nodos.

Recorrido BFS

Tomando como punto de inicio la Entrada Principal (A), el recorrido obtenido es:

A → B → D → C → E → G → F

Este algoritmo visita primero todos los nodos más cercanos y luego continúa con los siguientes niveles.

Aplicación de A*

Un caso donde utilizaríamos A* sería cuando un estudiante desea llegar desde la Entrada Principal hasta el Laboratorio de Informática por la ruta más rápida.

La heurística utilizada sería la **distancia aproximada en metros** desde cada ambiente hasta el laboratorio. De esta forma, el algoritmo puede estimar cuál es el mejor camino.

La ruta recomendada sería:

A → B → C

Comparación de los algoritmos

DFS: Es recomendable cuando se desea recorrer completamente el campus o explorar todas las rutas posibles.

BFS: Es la mejor opción cuando se busca el lugar más cercano, ya que visita primero los nodos vecinos.

A*: Es el algoritmo más conveniente para recomendar rutas, porque utiliza una heurística para encontrar un camino más eficiente.

Conclusión

Representar el campus mediante un grafo permite organizar fácilmente las conexiones entre los diferentes ambientes. Los algoritmos DFS, BFS y A* tienen aplicaciones distintas: DFS sirve para explorar todas las rutas, BFS para encontrar los lugares más cercanos y A* para recomendar el camino más eficiente. Gracias a estos algoritmos es posible mejorar la orientación de los estudiantes dentro del campus.

ALGORITMO DFS

DFS(nodo)

Marcar el nodo como visitado.

Mostrar el nodo.

Para cada vecino del nodo:

Si no ha sido visitado:

DFS(vecino)

ALGORITMO BFS

BFS(inicio)

Crear una cola.

Agregar el nodo inicial.

Marcarlo como visitado.

Mientras la cola no esté vacía:

Sacar el primer nodo.

Mostrar el nodo.

Agregar a la cola los vecinos que aún no fueron visitados.

ALGORITMO A*

AStar(inicio, destino)

Elegir el nodo inicial.

Calcular el costo del recorrido.

Calcular la heurística.

Elegir el nodo con el menor costo.

Repetir hasta llegar al destino.

Mostrar la ruta encontrada.

Actividad 4

Conclusión grupal

Como grupo, aprendimos que la recursividad y los grafos son herramientas fundamentales para resolver problemas reales de manera organizada y eficiente. La recursividad nos permitió comprender cómo un problema complejo puede dividirse en tareas más pequeñas hasta llegar a un caso base, aplicando el principio de divide y vencerás. Esto se evidenció tanto en el conteo de puntos de control como en el algoritmo Merge Sort, donde la solución se obtiene resolviendo primero subproblemas más simples.

Por otro lado, el uso de grafos nos permitió representar de forma clara las conexiones entre los diferentes ambientes del campus y aplicar algoritmos de búsqueda como DFS, BFS y A*. Cada uno de estos algoritmos tiene un propósito específico: explorar

todas las rutas, encontrar los lugares más cercanos o determinar el camino más eficiente. En conjunto, estos conceptos demostraron cómo las estructuras de datos y los algoritmos pueden utilizarse para modelar y resolver situaciones que se presentan en un entorno real, sentando además las bases para aplicaciones relacionadas con la inteligencia artificial.

Justificación de la solución y mejoras propuestas

La solución propuesta es pertinente para un entorno educativo porque representa un problema cercano a la realidad de una institución académica: la orientación y el desplazamiento dentro del campus. Mediante la utilización de grafos es posible modelar las conexiones entre los distintos ambientes, mientras que los algoritmos de búsqueda permiten encontrar recorridos adecuados según la necesidad del usuario. Además, la implementación de algoritmos recursivos fortalece la comprensión de conceptos fundamentales de programación y fomenta el desarrollo del pensamiento lógico y la resolución de problemas.

Organización de la entrega en GitHub y evidencias en el README

El repositorio de GitHub se organizará en carpetas para facilitar la navegación y el mantenimiento del proyecto. Se incluirá una carpeta para el código fuente y otra para la documentación del proyecto.

El archivo README incluimos una descripción general del trabajo, los objetivos del proyecto, los nombres de los integrantes del grupo, la explicación de cada actividad desarrollada, las instrucciones para ejecutar el código y una breve descripción de los algoritmos implementados (recursividad, Merge Sort, DFS, BFS y A*). Como evidencias se incluirán capturas de pantalla de la ejecución de los programas, el diagrama del grafo del campus, la traza de la recursividad, el proceso de ordenamiento con Merge Sort y el enlace al repositorio para que el docente pueda revisar el historial de cambios y la participación del equipo.